

SE 811: Software Maintenance

Toukir Ahammed

Software Reengineering

Software Reengineering

- Reengineering is the examination, analysis, and restructuring of an existing software system to reconstitute it in a new form and the subsequent implementation of the new form.
- The goal of reengineering is to:
 - understand the existing software system artifacts, namely, specification, design, implementation, and documentation; and
 - improve the functionality and quality attributes of the system (e.g., evolvability, performance, and reusability)

Software Reengineering

- Fundamentally, a new system is generated from an operational system, such that the target system possesses better quality factors.
- The desired software quality factors include reliability, correctness, integrity, efficiency, maintainability, usability, flexibility, testability, interoperability, reusability, and portability.
- In other words, reengineering is done to convert an existing “bad” system into a “good” system.

Software Reengineering

- Of course there are risks involved in this transformation, and the primary risks are:
 - (i) the target system may not have the same functionality as the existing system;
 - (ii) the target system may be of lower quality than the original system; and
 - (iii) the benefits are not realized in the required time frame

General Objectives of Reengineering

- Software systems are reengineered by keeping one or more of the following four general objectives in mind:
 - Improving maintainability
 - Migrating to a new technology
 - Improving quality
 - Preparing for functional enhancement.

General Objectives of Reengineering

Improving maintainability

- As systems grow and evolve, the cost of maintenance increases because changes become difficult and time consuming.
- Consequently, it is unrealistic to avoid reengineering in the long run.

General Objectives of Reengineering

Migrating to a new technology

- In the fast-paced IT industry, new— and sometimes cheaper—execution platforms include new features, which quickly make the current system outdated, and maybe more expensive.
- Compatibility of the newer system with the old one is likely to be an issue, because vendors have less motivation to provide support for older parts that become incompatible and more expensive.
- Moreover, as systems evolve, expertise of employees migrates to newer technologies, with fewer staff to maintain the old system.
- Consequently, organizations with perfectly working software that continues to meet their business needs are forced to migrate to a modern execution platform that includes newer hardware, operating system, and/or language.

General Objectives of Reengineering

Improving quality

- As time passes, users make increasingly more change requests to modify the system.
- Each change causes “ripple effects,” implying that one change causes more problems to be fixed.
- As the system is continually modified as a result of maintenance activities, the reliability of the software gradually decreases to an unacceptable level.
- Therefore, the system must be reengineered to achieve greater reliability.

General Objectives of Reengineering

Preparation for functional enhancement

- In general, reengineering is not performed to support more functionalities of a system; rather, as a preparatory step to enhance functionalities, a system is reengineered.
- For example, if the objective is to transform programs from a procedural to an object-oriented form to distribute them in a client-server architecture, then, at the same time, plan to reduce the maintenance costs by using a language such that the system will be more amenable to changes.

Reengineering Concepts

- Several concepts applied during software development are key to reengineering of software.
- For example, *abstraction* and *refinement* are key concepts used in software development, and both the concepts are equally useful in reengineering.
- Abstraction enables software maintenance personnel to reduce the complexity of understanding a system by:
 - (i) focusing on the more significant information about the system and
 - (ii) hiding the irrelevant details at the moment.
- On the other hand, refinement is the reverse of abstraction.

Reengineering Concepts

Principle of abstraction

- The level of abstraction of the representation of a system can be gradually increased by successively replacing the details with abstract information. By means of abstraction one can produce a view that focuses selected system characteristics by hiding information about other characteristics.

Principle of refinement.

- The level of abstraction of the representation of the system is gradually decreased by successively replacing some aspects of the system with more details.

Levels of Abstraction and Refinement

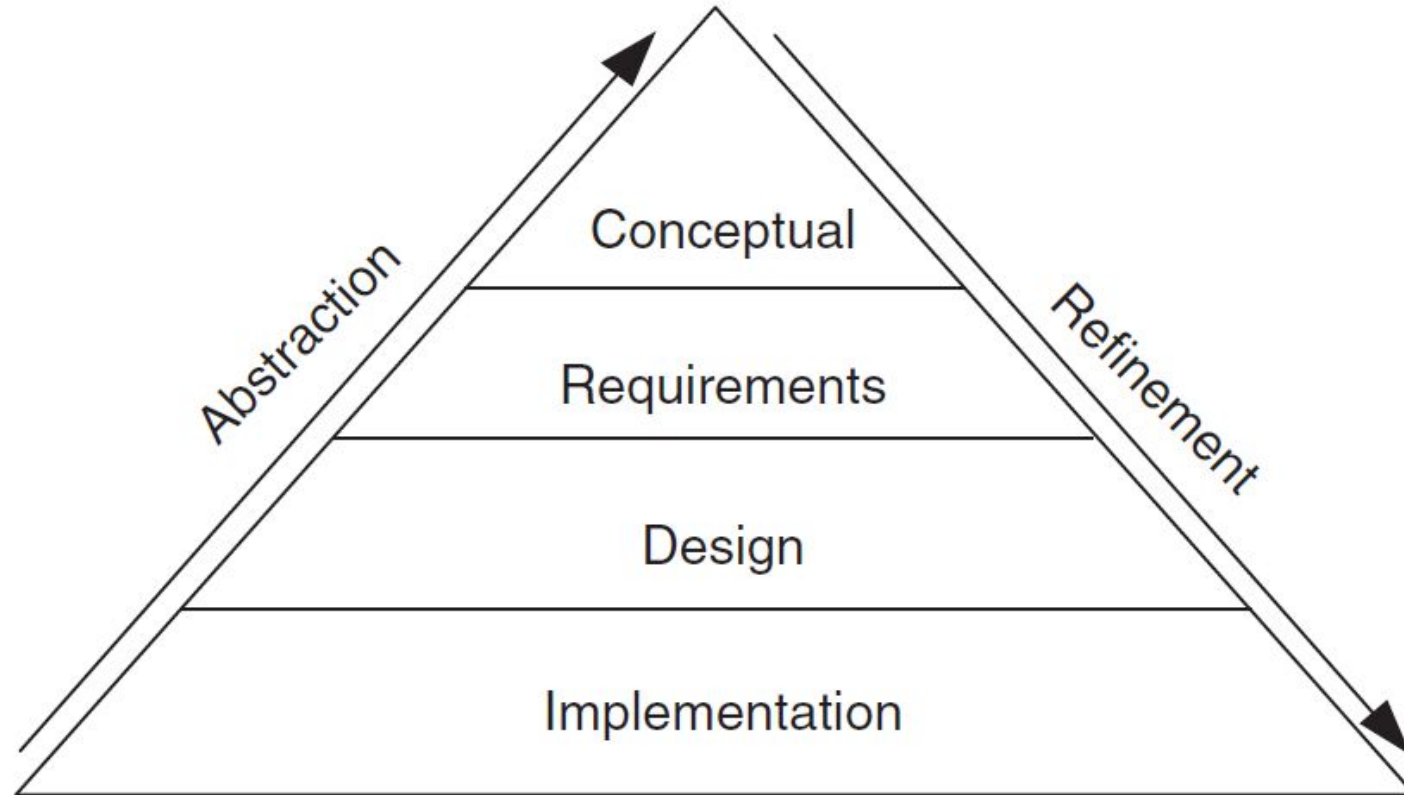


FIGURE 4.1 Levels of abstraction and refinement. From Reference 5. © 1992 IEEE

Levels of Abstraction and Refinement

Conceptual level.

- At the highest level of abstraction, the software is described in terms of very high-level concepts and its reason for existence (*why?*).
- In other words, this level addresses the “why” aspects of the system, by answering the question:

“Why does the system exist?”

Levels of Abstraction and Refinement

Requirements level.

- At this level, functional characteristics (*what?*) of the system are described at a high level, while leaving the details out.
- In other words, this level addresses the “what” aspects of the system by answering the question:

“What does the system do?”

Levels of Abstraction and Refinement

Design level.

- At the design-refinement level, system characteristics (*what and how?*), namely, major components, the architectural style putting the components together, the interfaces among the components, algorithms, major internal data structures, and databases are described in detail.
- In other words, this level addresses more of “what” and “how” aspects of the system by answering the questions:
 - (i) “*What are the characteristics of the system?*” and
 - (ii) “*How is the system going to possess the characteristics to deliver the functionalities?*”

Levels of Abstraction and Refinement

Implementation level.

- This is the lowest level of abstraction in the hierarchy.
- At this level, the system is described at a very low level in terms of implementation details in a high-level language.
- In other words, this level addresses “how” exactly the system is implemented.

In summary, the refinement process can be represented as why? → what? → what and how? → how? and the abstraction process can be represented as how? → what and how? → what? → why?

Cycle of Abstraction and Refinement

- A concept, a requirement, a design, and an implementation of a program usually denote different levels of abstraction. Moving from one level to another involves a process of crossing levels of abstraction.
- Usually a specification is more abstract than its implementation; therefore, the cycle of abstraction and refinement can be represented as follows: *concrete* → *more abstract* → *abstract* → *highly abstract* → *abstract* → *less abstract* → *concrete*.
- Abstraction and refinement are important concepts, and these are useful in reengineering as well as in forward engineering.

Forward Engineering

- A new software is created by going downward from the top, highest level of abstraction to the bottom, lowest level.
- This downward movement is known as forward engineering.
- Forward engineering follows a sequence of activities:
 - formulating concepts about the system to identifying requirements to designing the system to implementing the design.

Reverse Engineering

- On the other hand, the upward movement through the layers of abstractions is called reverse engineering.
- Reverse engineering of software systems is a process comprising the following steps:
 - (i) analyze the software to determine its components and the relationships among the components and
 - (ii) represent the system at a higher level of abstraction or in another form.

Reverse Engineering

- Some examples of reverse engineering are:
 - (i) decompilation, in which object code is translated into a high-level program;
 - (ii) architecture extraction, in which the design of a program is derived;
 - (iii) document generation, in which information is produced from, say, source code, for better understanding of the program; and
 - (iv) software visualization, in which some aspect of a program is depicted in an abstract way.

Reengineering Concepts

Principle of alteration.

- The making of some changes to a system representation is known as alteration.
- Alteration does not involve any change to the degree of abstraction, and it does not involve modification, deletion, and addition of information.

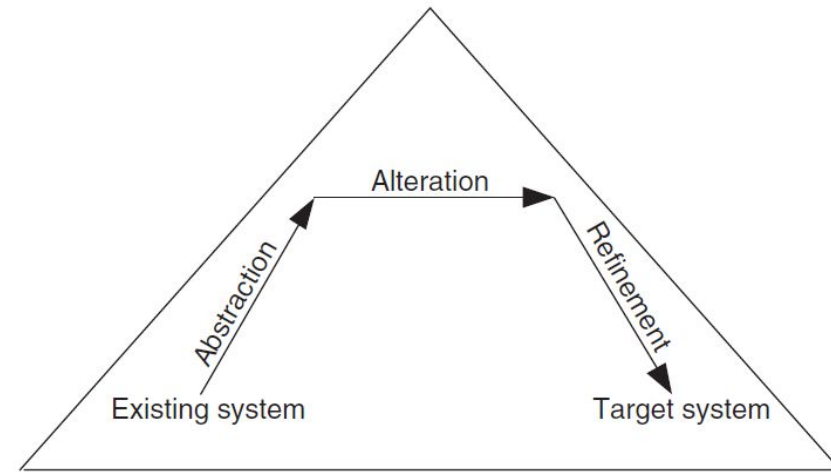


FIGURE 4.2 Conceptual basis for the reengineering process. From Reference 5. © 1992 IEEE

Reengineering Process

- The reengineering process accepts as input the existing code of a system and produces the code of the renovated system.
- On the one hand, the reengineering process may be as straightforward as translating with a tool the source code from the given language to source code in another language.
- For example, a program written in BASIC can be translated into a new program in C.

Reengineering Process

- On the contrary, the reengineering process may be very complex as explained below:
 - recreate a design from the existing source code;
 - find the requirements of the system being reengineered;
 - compare the existing requirements with the new ones;
 - remove those requirements that are not needed in the renovated system;
 - make a new design of the desired system; and
 - code the new system.

Software Reengineering Definitions

- Software reengineering is the analysis and alteration of an operational system to represent it in a new form and to obtain a new implementation from the new form. Here, a new form means a representation at a higher level of abstraction.
- Reengineering of a software system is a process for creating a new software from an existing software so that the new system is better than the original system in some ways.
- Reengineering of a software system is an activity that:
 - (i) improves the comprehension of the software system or
 - (ii) raises the quality levels of the software, namely, performance, reusability, and maintainability.

Software Reengineering Definitions

- In summary, it is evident that reengineering entails:
 - (i) the creation of a more abstract view of the system by means of some reverse engineering activities;
 - (ii) the restructuring of the abstract view; and
 - (iii) implementation of the system in a new form by means of forward engineering activities.

Software Reengineering Definitions

- This process is formally captured by Jacobson and Lindstrom with the following expression:

$$\textit{Reengineering} = \textit{Reverse engineering} + \Delta + \textit{Forward engineering}$$

- *Reverse engineering* is an activity to create an easier to understand and more abstract form of the system.
- *Forward engineering* is the traditional process of moving from a high-level abstraction and logical, implementation-independent design to the physical implementation of the system.

Software Reengineering Definitions

- Δ captures alterations made to the original system.
 - change in functionality and
 - change in implementation technique.
- A change in functionality comes from a change in the business rules.
- Moreover, change of functionality does not affect how the system is implemented, that is, how forward engineering is carried out.
- Often, reengineering is associated with the introduction of a new development technology, for example, model-driven engineering.
- A variant of reengineering in which the transformation is driven by a major technology change is called *migration*.

Rehosting

- Another common term used by practitioners of reengineering is *rehosting*.
- Rehosting means reengineering of source code without addition or reduction of features in the transformed targeted source code.
- Rehosting is most effective when the user is satisfied with the system's functionality, but looks for better qualities of the system.
- Examples of better qualities are improved efficiency of execution and reduced maintenance costs.

Software Reengineering Model

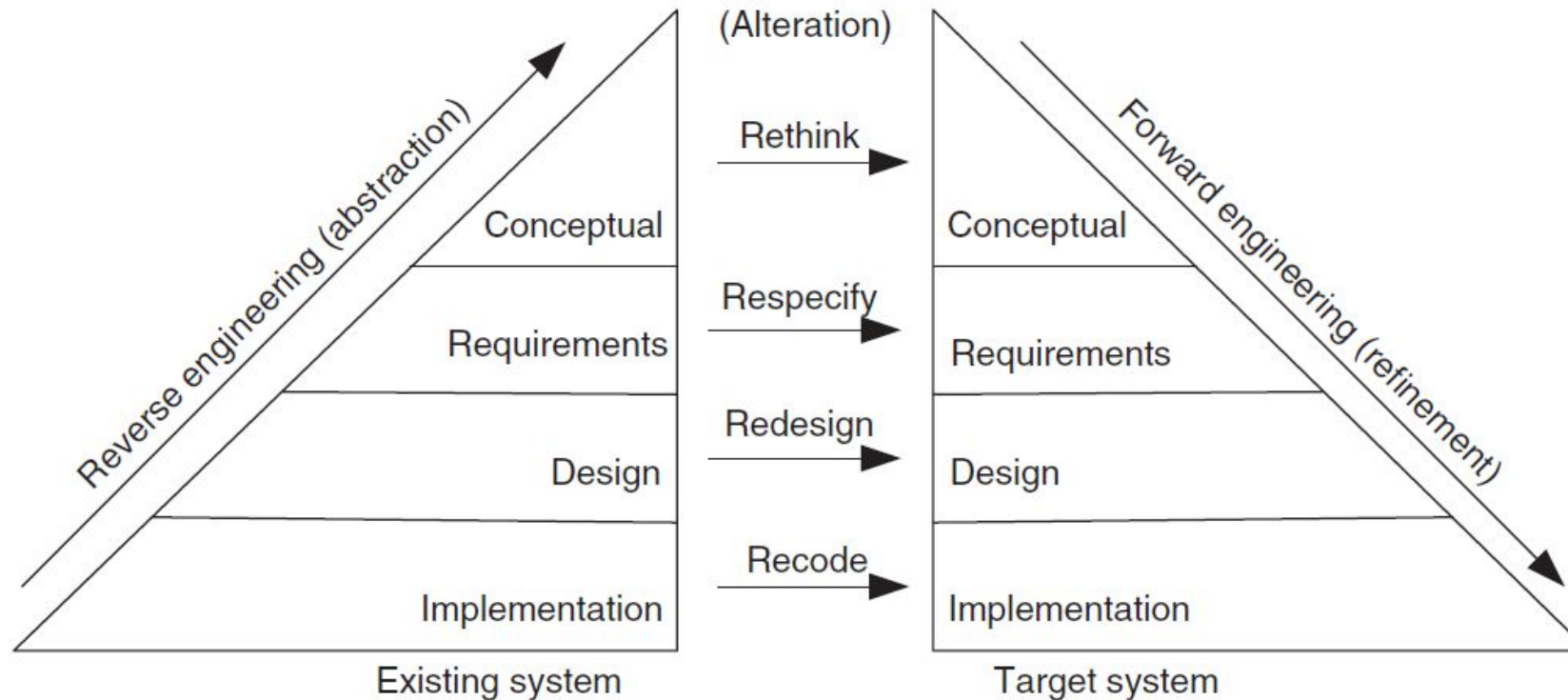


FIGURE 4.3 General model of software reengineering. From Reference 5. © 1992 IEEE

An existing system can be reengineered by following one of four paths.

Types of Changes

Based on the type of changes required, system characteristics are divided into groups:

- Rethink
- Respecify,
- Redesign, and
- Recode.

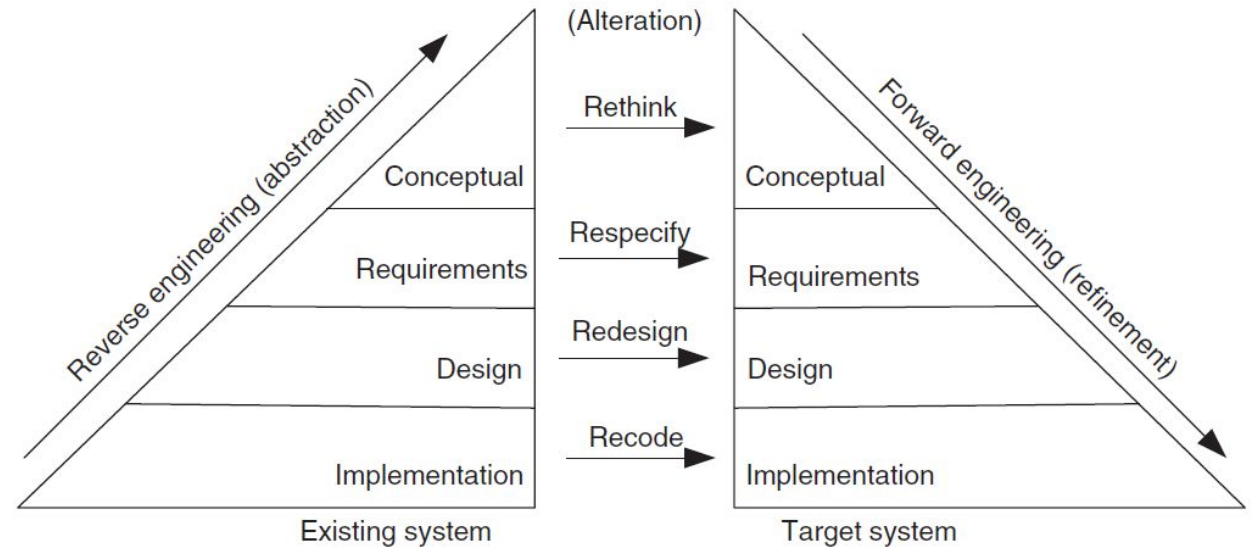


FIGURE 4.3 General model of software reengineering. From Reference 5. © 1992 IEEE

Types of Changes

Recode

- Implementation characteristics of the source program are changed by recoding it.
- Source code level changes are performed by means of
 - *rephrasing*: keeps the program in the same language
 - *program translation*: a program is transformed into a program in a different language

Types of Changes

Recode

- Examples of translation scenarios are:
 - *compilation*: one transforms a program written in a high-level language into assembly or machine code
 - *decompilation*: a form of transformation in which high-level source code is discovered from an executable program
 - *migration*: a program is transformed into a program in another language while retaining the program's abstraction level.

Types of Changes

Recode

- Examples of rephrasing scenarios are
 - *normalization*: reduces a program to a program in a sublanguage, that is to a subset of the language, with the purpose of decreasing its syntactic complexity. (e.g., Elimination of GOTO in a program.
 - *optimization*: a transformation that improves the execution time or space performance of a program.
 - *refactoring*: a transformation that improves the design of a program by means of restructuring to better understand the new program.

Types of Changes

Redesign

- The design characteristics of the software are altered by redesigning the system.
- Common changes to the software design include:
 - (i) restructuring the architecture;
 - (ii) modifying the data model of the system; and
 - (iii) replacing a procedure or an algorithm with a more efficient one.

Types of Changes

Respecify

- This means changing the requirement characteristics of the system in two ways:
 - (i) change the form of the requirements: changing only the form of existing requirements, that is, taking the informal requirements expressed in a natural language and generating a formal specification in a formal description language (e.g., Specification and Description Language (SDL) or Unified Modeling Language (UML)).
 - (ii) change the scope of the requirements: changes includes such changes as adding new requirements, deleting some existing requirements, and altering some existing requirements.

Types of Changes

Rethink

- Rethinking a system means manipulating the concepts embodied in an existing system to create a system that operates in a different problem domain.
- It involves changing the conceptual characteristics of the system, and it can lead to the system being changed in a fundamental way.
- Moving from the development of an ordinary cellular phone to the development of smartphone system is an example of Rethink.

Software Reengineering Strategies

Three strategies that specify the basic steps of reengineering are:

- *rewrite*: based on the principle of alteration
- *rework*: based on the principles of abstraction, alteration, and refinement
- *replace*: based on the principles of abstraction and refinement.

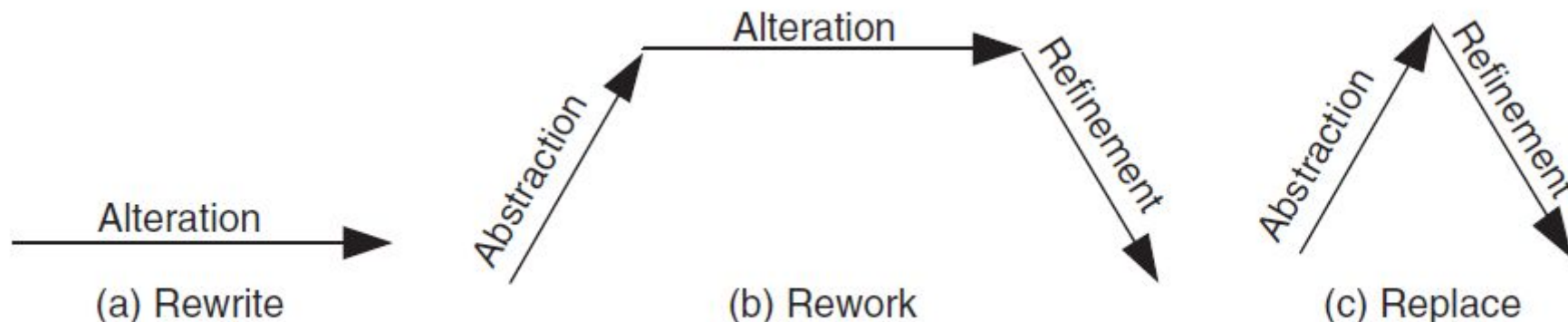
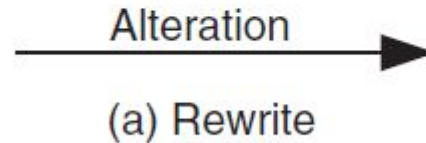


FIGURE 4.5 Conceptual basis for reengineering strategies. From Reference 5. © 1992 IEEE

Software Reengineering Strategies

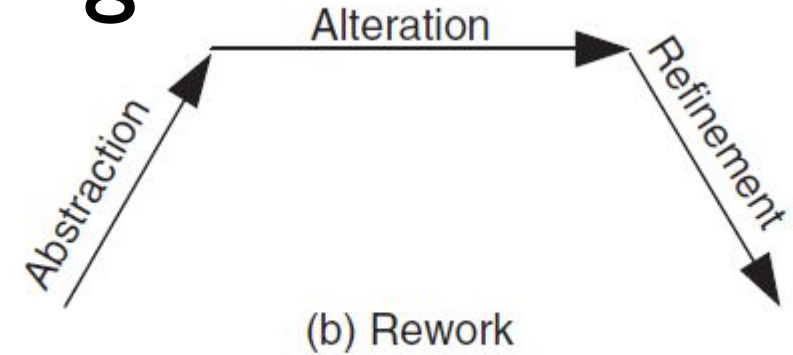
Rewrite strategy



- By means of alteration, an operational system is transformed into a new system while preserving the abstraction level of the original system.
- For example, the Fortran code of a system can be rewritten in the C language.

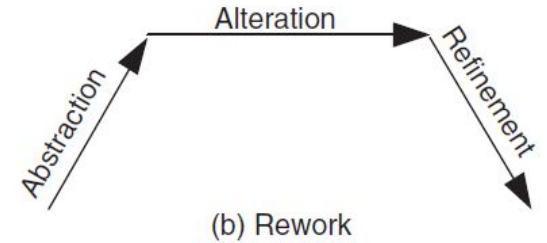
Software Reengineering Strategies

Rework strategy



- First, by means of the principle of abstraction, obtain a system representation with less details than what is available at a given level. (e.g., one can create an abstraction of source code in the form of a high-level design.)
- Next, the reconstructed system model is transformed into the target system representation, by means of alteration, without changing the abstraction level.
- Finally, by means of refinement, an appropriate new system representation is created at a lower level of abstraction.

Software Reengineering Strategies

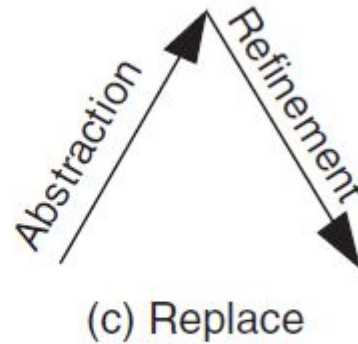


Rework strategy

- Let the goal of a reengineering project be to restructure the control flow of a program.
 - replace the unstructured control flow constructs, namely GOTOs, with more commonly used structured constructs, say, a “for” loop.
- A classical, rework strategy-based approach to doing that is as follows:
 - Application of abstraction: By parsing the code, generate a control flow graph (CFG) for the given system.
 - Application of alteration: Apply a restructuring algorithm to the CFG to produce a structured CFG.
 - Application of refinement: Translate the new, structured CFG back into the original programming language.

Software Reengineering Strategies

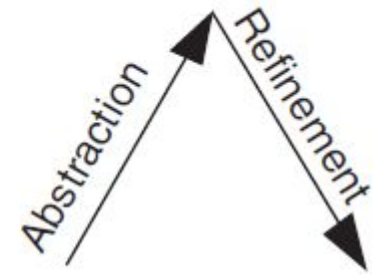
Replace strategy



- To change a certain characteristic of a system:
 - (i) the system is reconstructed at a higher level of abstraction by hiding the details of the characteristic and
 - (ii) a suitable representation for the target system is generated at a lower level of abstraction by applying refinement.

Software Reengineering Strategies

Replace strategy



(c) Replace

- Let us reconsider the GOTO example.
- By means of abstraction, a program is represented at a higher level without using control flow concepts.
- For instance, a module's behavior can be described by its net effect, with no mention of control flow.
- Next, by means of refinement, the system is represented at a lower level of abstraction with a new structured control flow.
- In summary, the original unstructured control flow is replaced with a structured control flow.

Reengineering Variations

If A is the abstraction level of the representation of the system to be reengineered and the plan is to make a B type of change, can I use strategy C ?

Starting Abstraction Level	Type Change	Reengineering Strategy		
		Rewrite	Rework	Replace
Implementation level	Recode	Yes	Yes	Yes
	Redesign	Bad	Yes	Yes
	Respecify	Bad	Yes	Yes
	Rethink	Bad	Yes*	Yes*
Design level	Recode	No	No	No
	Redesign	Yes	Yes	Yes
	Respecify	Bad	Yes	Yes
	Rethink	Bad	Yes*	Yes*
Requirement level	Recode	No	No	No
	Redesign	No	No	No
	Respecify	Yes	Yes	Yes
	Rethink	Bad	Yes*	Yes*
Conceptual level	Recode	No	No	No
	Redesign	No	No	No
	Respecify	No	No	No
	Rethink	Yes	Yes*	Yes*

Source: From Reference 5. © 1992 IEEE.

Yes—One can produce a target system.

Yes*—Same as Yes, but the starting degree of abstraction is lower than the uppermost degree of abstraction within the conceptual abstraction level.

No—One cannot start at abstraction level A , make B type of changes by using strategy C , because the starting abstraction level is higher than the abstraction level required by the particular type of change.

Bad—A target system can be created, but the likelihood of achieving a good result is low.

Software Reverse Engineering

- Chikofsky and Cross II defined *reverse engineering* as a process to
 - (i) identify the components of an operational software;
 - (ii) identify the relationships among those components; and
 - (iii) represent the system at a higher level of abstraction or in another form.
- In other words, by means of reverse engineering one derives information from the existing software artifacts and transforms it into abstract models to be easily understood by maintenance personnel.

When Reverse Engineering is Required?

The factors necessitating the need for reverse engineering are as follows:

- The original programmers have left the organization.
- The language of implementation has become obsolete, and the system needs to be migrated to a newer one.
- There is insufficient documentation of the system.
- The business relies on software, which many cannot understand.
- The company acquired the system as part of a larger acquisition and lacks access to all the source code.
- The system requires adaptations and/or enhancements.
- The software does not operate as expected.

Software Reverse Engineering

Reverse engineering is performed to achieve two key objectives:

- *redocumentation of artifacts*: aims at revising the current description of components or generating alternative views at the same abstraction level (e.g., pretty printing and drawing CFGs)
- *design recovery*: creates design abstractions from code, expert knowledge, and existing documentation (identifying meaningful higher-level abstractions beyond those obtained directly by examining the system itself).

Software Reverse Engineering

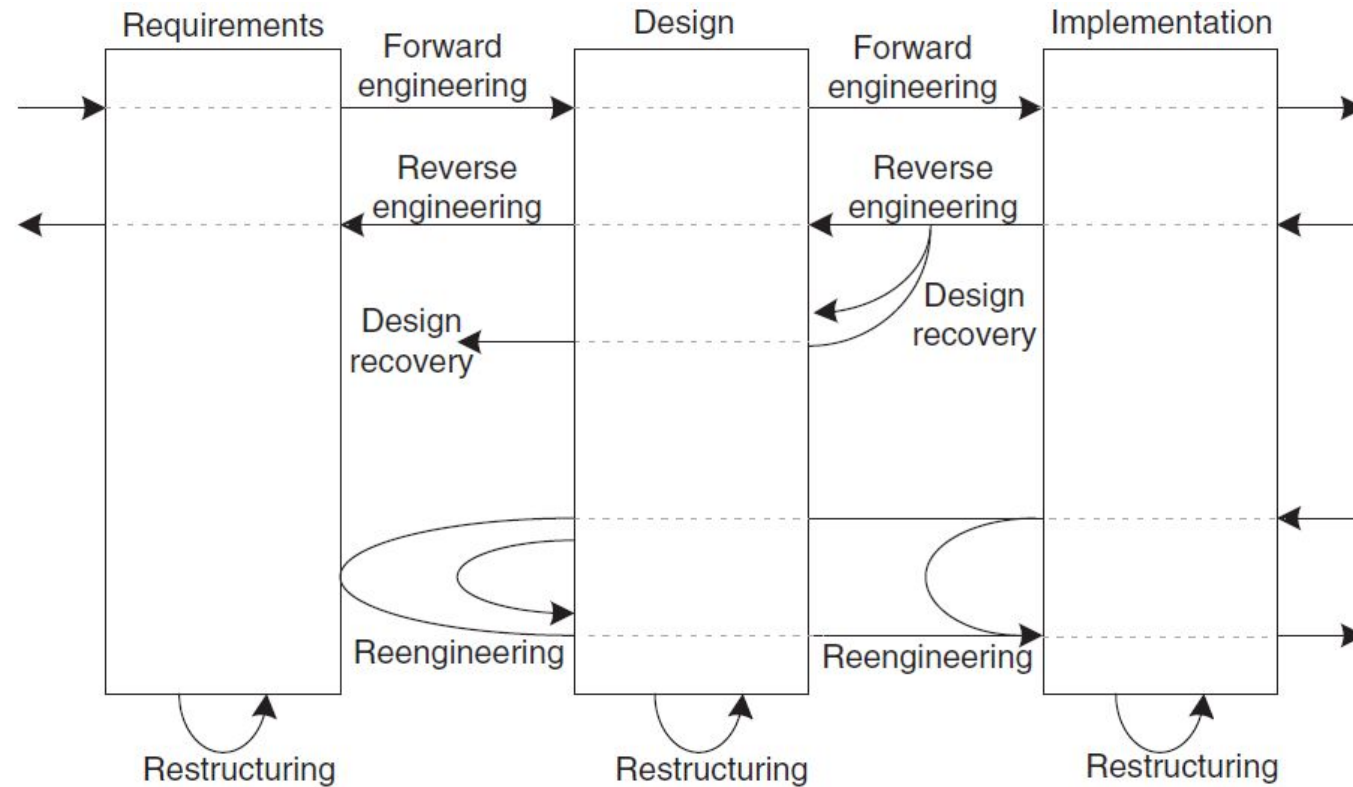


FIGURE 4.10 Relationship between reengineering and reverse engineering. From Reference 6. © 1990 IEEE

Objectives Of Reverse Engineering

1. Generating alternative views,
2. Recovering lost information,
3. Synthesizing higher Levels of abstractions,
4. Detecting side effects,
5. Facilitating reuse, and
6. Coping with complexity.

Objectives Of Reverse Engineering

Generating alternative views

- Redocumentation tools can be used to provide alternative documentation such as data flow diagrams, control flow diagrams and entity-relationship diagrams in addition to the existing documentation.
- This is a means whereby other views of the system can be obtained.
- For example, data flow diagrams portray the system from the point of view of data flow within the system and outside.
- Control flow diagrams, on the other hand, show the system from the perspective of the flow of control between the different components.

Objectives Of Reverse Engineering

Recovering lost information

- With time, a system undergoes a series of changes.
- Because of such things as management pressure and time constraints, the corresponding documentation for the requirements specification and design may not be kept up to date and may not even exist.
- This makes the code the only source of information about the system.
- Reverse engineering tools allow this information (requirements specification and design) to be recovered.
- Example: recovering specification would be in a specification language such as Z, and representing design as data flow diagrams, control flow diagrams, and entity-relationship diagrams.

Objectives Of Reverse Engineering

Detecting side effects

- In cases where the maintainer lacks a global view of the system, undesired side effects are caused and anomalies go unnoticed.
- Reverse engineering tools can make the general architecture of the system visible, thereby making it easier to predict the effect of change and detect logic and data flow problems.

Objectives Of Reverse Engineering

Facilitating reuse

- The use of existing program components can lead to an increase in productivity and improvement in product quality, the concept of reuse has increasingly become popular amongst software engineers.
- Success in reusing components depends in part on their availability.
- Reverse engineering tools and methods offer the opportunity to access and extract reusable program components.

Objectives Of Reverse Engineering

Cope with Complexity

- One of the major problems with legacy systems is that as they evolve, their complexity increases.
- In the event of a modification, this complexity must be dealt with by abstracting system information relevant to the change and ignoring that which is irrelevant.
- Reverse engineering tools together with CASE tools provide the maintainer with some form of automated support for both function and data abstractions

Software Reverse Engineering

Reverse engineering has been effectively applied in the following problem areas:

- redocumenting programs;
- identifying reusable assets;
- discovering design architectures;
- recovering design patterns;
- building traceability between code and documentation
- finding objects in procedural programs
- deriving conceptual data models
- detecting duplications and clones
- cleaning up code smells
- aspect-oriented software development
- computing change impact;
- transforming binary code into source code;
- redesigning user interfaces;
- parallelizing largely sequential programs;
- translating a program to another language;
- migrating data;
- extracting business rules;
- wrapping legacy code;
- auditing security and vulnerability;
- extracting protocols of network applications

Techniques Used for Reverse Engineering

In order to extract information which is not explicitly available in source code, automated analysis techniques are used. The well-known analysis techniques that facilitate reverse engineering are:

- Lexical analysis
- Syntactic analysis
- Control flow analysis
- Data flow analysis
- Program slicing
- Visualization
- Program metrics

Lexical Analysis

- Lexical analysis is the process of decomposing the sequence of characters in the source code into its constituent lexical units.
- A program performing lexical analysis is called a lexical analyzer, and it is a part of a programming language's compiler.

Lexical Analysis

- breaking syntaxes into a series of tokens, by removing any whitespace or comments in the source code.
- keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.

```
int value = 100;
```

int (*keyword*)

value (*identifier*)

= (*operator*)

100 (*constant*)

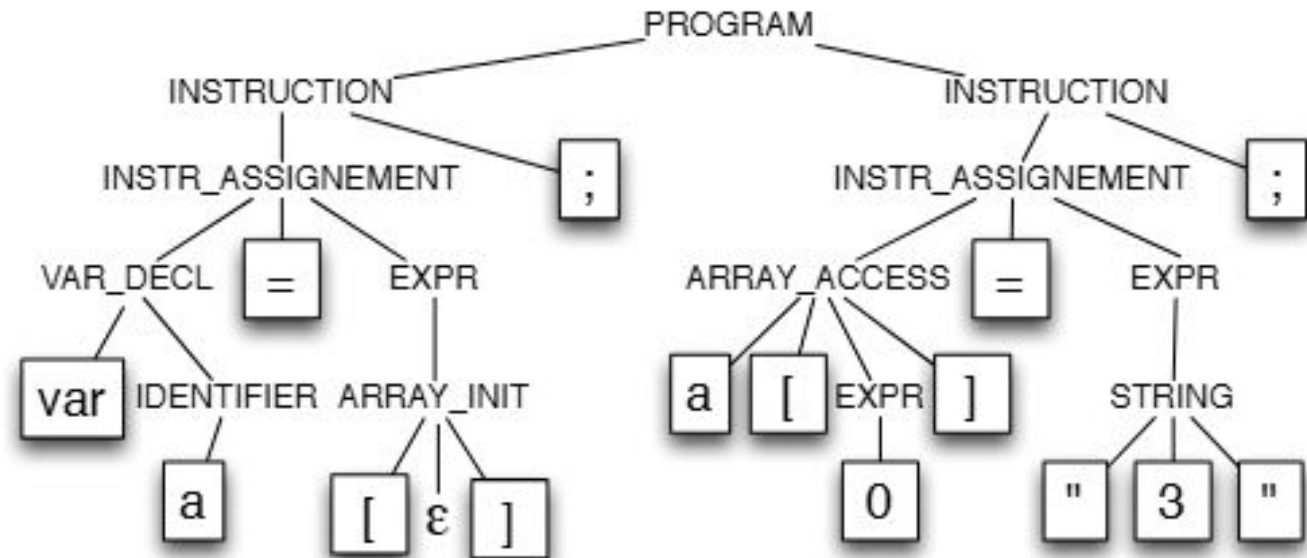
; (*symbol*)

Syntax Analysis

- Two types of representations are used to hold the results of syntactic analysis: parse tree and abstract syntax tree.
- A parse tree contains details unrelated to actual program meaning, such as the punctuation, whose role is to direct the parsing process.
- Removal of extraneous details produces a structure called an Abstract Syntax Tree (AST).
- An AST contains just those details that relate to the actual meaning of a program.

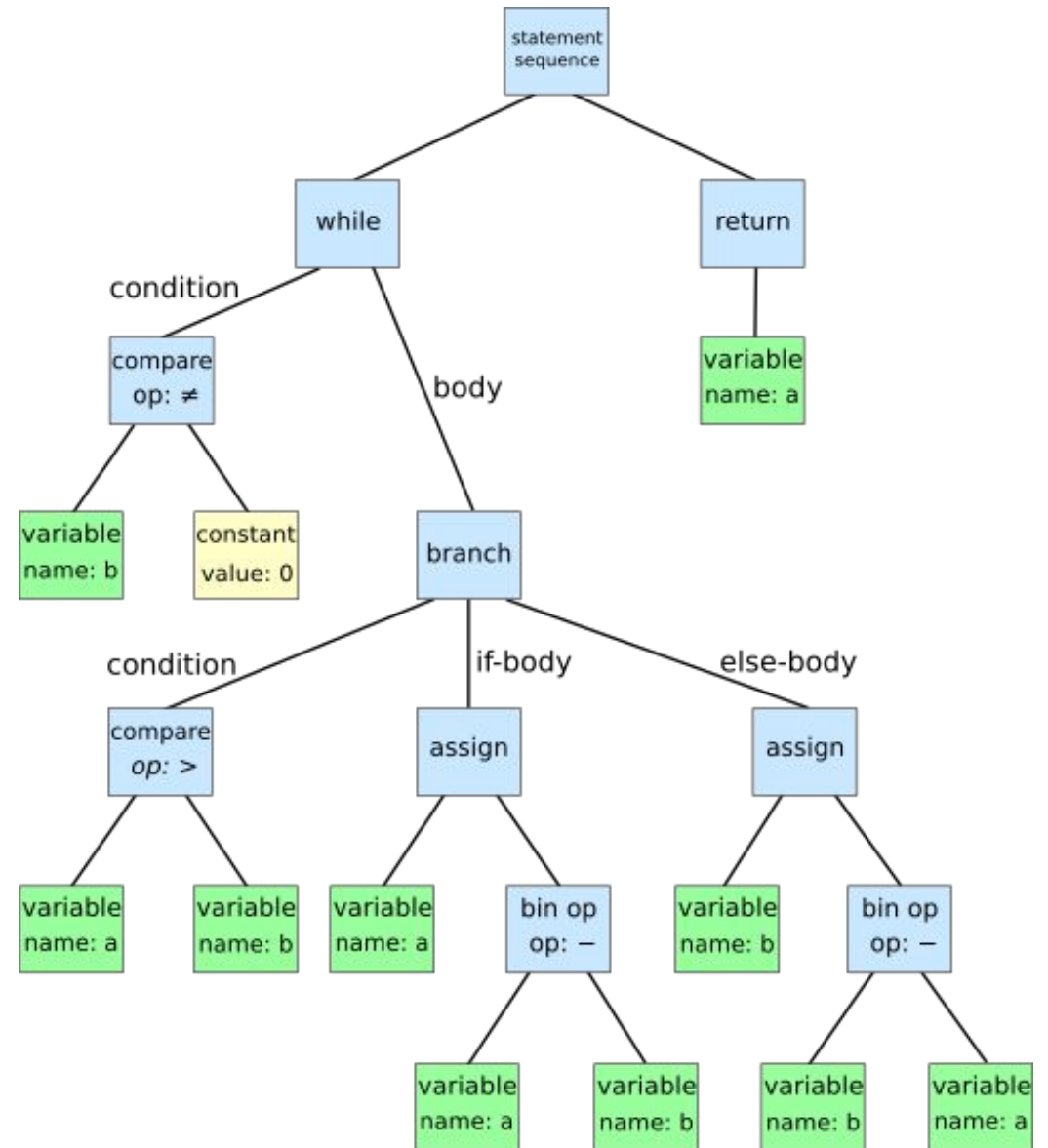
A parse tree

`var a = [] ;`
`a[0] = 3 ;`



An abstract syntax tree

```
while b != 0:  
    if a > b:  
        a := a - b  
    else:  
        b := b - a  
return a
```



Control Flow Analysis

The two kinds of control flow analysis are:

- Intraprocedural: It shows the order in which statements are executed within a subprogram.
- Interprocedural: It shows the calling relationship among program units.

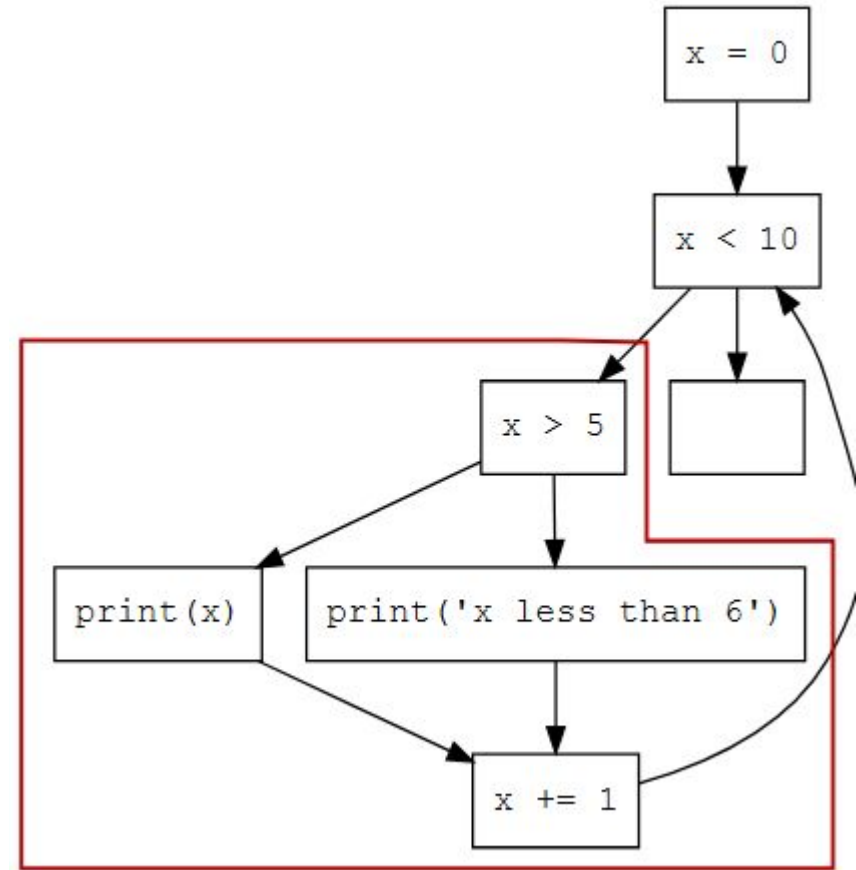
Control Flow Analysis

Intraprocedural analysis:

- The idea of basic blocks is central to constructing a CFG.
- A basic block is a maximal sequence of program statements such that execution enters at the top of the block and leaves only at the bottom via a conditional or an unconditional branch statement.
- A basic block is represented with one node in the CFG, and an arc indicates possible flow of control from one node to another.
- A CFG can directly be constructed from an AST by walking the tree to determine basic blocks and then connecting the blocks with control flow arcs.

A control flow graph

```
x = 0
while x < 10:
    if x > 5:
        print(x)
    else:
        print('x less
than 6')
    x += 1
```



Control Flow Analysis

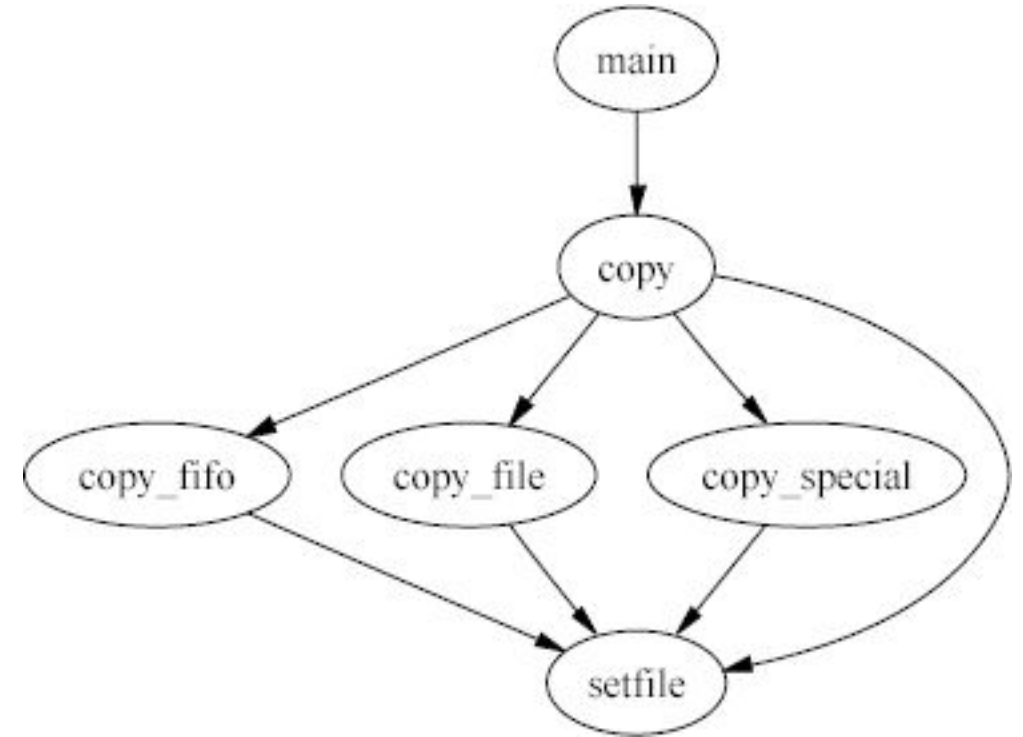
Interprocedural analysis:

- Interprocedural analysis is performed by constructing a call graph.
- Calling relationships between subroutines in a program are represented as a call graph which is basically a directed graph.
- Specifically, a procedure in the source code is represented by a node in the graph, and the edge from node f to g indicates that procedure f calls procedure g .

Call Graph

Call graphs can be static or dynamic.

- A dynamic call graph is an execution trace of the program. Thus, a dynamic call graph is exact, but it only describes one run of the program.
- On the other hand, a static call graph represents every possible run of the program.



Data Flow Analysis

- Control flow analysis cannot answer the question: Which program statements are likely to be impacted by the execution of a given assignment statement?
- To answer this kind of questions, an understanding of definitions (def) of variables and references (uses) of variables is required.
- Data flow analysis (DFA) concerns how values of defined variables flow through and are used in a program

Data Flow Analysis

- CFA can detect the possibility of loops, whereas DFA can determine data flow anomalies.
- One example of data flow anomaly is that an undefined variable is referenced.
- Another example of data flow anomaly is that a variable is successively defined without being referenced in between.
- DFA enables the identification of code that can never execute, variables that might not be defined before they are used, and statements that might have to be altered when a bug is fixed.

Program Slicing

- Originally introduced by Mark Weiser, program slicing has served as the basis of numerous tools.
- A program slice is a portion of a program with an execution behavior identical to the initial program with respect to a given criterion, but may have a reduced size.
- In Weiser's definition, a slicing criterion of a program P is $S < p; v >$ where p is a program point and v is a subset of variables in P .

Program Slicing

- The slice of a program for a given variable at a given line of code is the portion of the program that gives a value to the variable at that point.
- Therefore, if one determines during debugging that the value of a variable at a specific line is incorrect, one may look at the corresponding program slice to find the faulty code.

Program Slicing Example

```
[1]      int i;  
[2]      int sum = 0;  
[3]      int product = 1;  
[4]      for(i = 0; ((i < N) && (i % 2 == 0)); i++) {  
[5]          sum = sum + i;  
[6]          product = product * i;  
[7]      }  
[7]      printf("Sum = ", sum);  
[8]      printf("Product = ", product);
```

FIGURE 4.11 A block of code to compute the sum and product of all the even integers in the range $[0, N)$ for $N \geq 3$

Backward Slicing Example

Backward slices answer the question “What program components might effect a selected computation?”

```
[1]      int i;  
[2]      int sum = 0;  
[4]      for(i = 0; ((i < N) && (i % 2 == 0)); i++) {  
[5]          sum = sum + i;  
          }  
[7]      printf("Sum = ", sum);
```

FIGURE 4.12 The backward slice of code obtained from Figure 4.11 by using the criterion $S < [7]; \text{sum} >$

A backward slice with respect to a variable v and a given point p comprises all instructions and predicates which affect the value of v at point p .

Forward Slicing Example

“What program components might be effected by a selected computation?”

```
[3]    int product = 1;
[4]    for(i = 0; ((i < N) && (i % 2 == 0)); i++) {
[6]        product = product * i;
      }
[8]    printf("Product = ", product);
```

FIGURE 4.13 The forward slice of code obtained from Figure 4.11 by using the criterion $S < [3]; \text{product} >$

With respect to a variable v and a point p in a program, a forward slice comprises all the instructions and predicates which may depend on the value of v at p .